

Reducing Floats mod p

Algebraic FPSan as a residue semantics

Benoit Jacob
benoit.jacob@amd.com

Abstract

FPSan-style instrumentation replaces each floating-point value by a same-width fingerprint. For a source format of width w , Triton’s FPSan, which this repository implements as `Semantics::Triton`, uses a tuned set-theoretic scramble of source-format bit patterns into the ring $\mathbb{Z}/2^w\mathbb{Z}$. That scramble is extremely useful, but it is not a homomorphism from the original numeric values, so identities such as $2 + 2 = 4$ or $x + x = 2x$ need not hold.

The algebraic semantics in `hip-fpsan` replace the scramble by an actual homomorphism on the finite values. The observation is simple: every finite binary floating-point value is a dyadic rational, hence an element of $\mathbb{Z}[1/2]$. For every odd prime p , reduction modulo p gives a ring homomorphism

$$\mathbb{Z}[1/2] \longrightarrow \mathbb{F}_p.$$

The Field-family semantics are built from this map. The `SophieGermainRing` and `PythagoreanRing` semantics replace the field by carefully chosen composite residue rings, giving up fieldness in exchange for exact exponential/logarithmic and trigonometric identities.

1 What FPSan is trying to test

Suppose a numerical kernel is rewritten for performance. A reduction may be reassociated, an expression may be factored, an FMA may replace a multiply and an add, or a matrix intrinsic may change the accumulation order. Comparing floating-point results bit-for-bit is too strict: a value-preserving algebraic rewrite can change rounding. Tolerance tests, on the other hand, are often too weak and too problem-specific.

FPSan answers a different question. It asks whether two computations have the same exact algebraic fingerprint under a chosen abstract semantics. These rewritten semantics are intentionally deterministic, compact, and invariant under the ring laws. They are not a reference implementation of the source floating-point arithmetic.

The invariant throughout this document is width preservation. If the original floating-point type has w bits, then every FPSan semantics discussed here, including Triton’s semantics and the algebraic variants, stores its fingerprint in exactly w bits.

The central design choice is the leaf map: how a source floating-point value is sent into the rewritten arithmetic.

2 Triton’s scramble and the transported-arithmetic caveat

Triton’s FPSan uses a bijection

$$E : \{\text{source-format bit patterns of width } w\} \longrightarrow \mathbb{Z}/2^w\mathbb{Z}$$

with special tuning. The construction itself is mostly indifferent to the details of IEEE-754: what matters is that the sanitizer receives fixed-width bit patterns, and later, for the algebraic variants, that finite binary floating-point values are dyadic rationals. Writing 0 and 1 for the bit patterns of those values, and writing $-x$ for source-format sign negation of a bit pattern x , Triton’s tuning gives $E(0) = 0$, $E(1) = 1$, and $E(-x) = -E(x)$ in the target ring. After applying E , arithmetic is performed with ring operations in $\mathbb{Z}/2^w\mathbb{Z}$, with special handlers for operations such as division, exponentials, and trigonometry.

Because E is a bijection, one can always define an artificial operation on bit patterns by transporting the target-ring operation back through E :

$$x \boxplus_E y = E^{-1}(E(x) + E(y)).$$

With respect to $\boxplus_E$, the map E is a homomorphism by construction. That statement is true but not the statement a compiler engineer usually wants: $\boxplus_E$ is not floating-point addition, nor exact rational addition of the represented values. It is the arithmetic manufactured by the embedding.

The visible consequence is immediate. If u is the source-format bit pattern of 2.0, then typically

$$E(u) + E(u) \neq E(\text{bit pattern of } 4.0).$$

So Triton FPSan sees reassociation and distributivity, but it does not see ordinary value facts such as $2 + 2 = 4$, $x + x = 2x$, or $x/x = 1$ for every nonzero x .

The algebraic modes in this repository pursue the opposite leaf-map choice: keep the fingerprint compact, but make the leaf map a genuine homomorphism of the exact finite values.

3 Finite binary floats live in $\mathbb{Z}[1/2]$

A finite binary floating-point value can be written as

$$x = 2^e M,$$

where $M \in \mathbb{Z}$ is a signed integer significand and $e \in \mathbb{Z}$. Normal and subnormal values both have this form, after absorbing the sign into M . Thus every finite value lies in

$$\mathbb{Z}[1/2] = \left\{ \frac{a}{2^k} : a \in \mathbb{Z}, k \geq 0 \right\}.$$

This is the localization of $\mathbb{Z}$ obtained by inverting 2.

The maximal ideals of $\mathbb{Z}[1/2]$ are exactly the ideals generated by odd primes. Equivalently, an odd prime p is precisely a prime modulo which 2 remains invertible. We will write

$$\phi_p : \mathbb{Z}[1/2] \rightarrow \mathbb{F}_p, \quad \phi_p(2^e M) = 2^e (M \bmod p) \pmod{p}$$

for the resulting reduction homomorphism. For negative e , 2^e means $(2^{-1})^{-e}$ in $\mathbb{F}_p$. This is the only algebraic fact about finite binary floats that the Field-family semantics need.

This is the mathematical core of the Field-family semantics. The fingerprint of a finite value is the residue of its exact dyadic value. Addition, subtraction, and multiplication are the field operations. The faithful Field variants use field inversion for division; **FieldFast** keeps the same finite-value fingerprints and the same additive/multiplicative arithmetic, but treats nontrivial division and roots as tagged operations for speed. The cheap division cases $0/x = 0$ and $x/1 = x$ are still preserved.

4 Compactness, sentinels, and non-injectivity

By width preservation, a source format of width w gives only 2^w fingerprint codes. Let N be the finite residue modulus used by an algebraic semantics; for example, in the `Semantics::Field` case with target $\mathbb{F}_p$, $N = p$, while in the composite ring cases $N = pd$. The finite residues use the codes $0, \dots, N - 1$, and two further codes are reserved for non-finite values:

$$N \quad \text{for infinity}, \quad N + 1 \quad \text{for NaN}.$$

Thus the finite modulus must satisfy $N \leq 2^w - 2$.

This compactness has a price: the algebraic map is not injective. It cannot be, because many finite floats must share a finite set of residues. This is unlike Triton’s bit-pattern scramble, which is a bijection before arithmetic starts.

The following fact is what makes that compromise much less dangerous at the most important point.

Proposition 1 (Zero is not a finite collision). *Let N be odd, and suppose every finite value of the source format can be written as $2^e M$ with $|M| < N$. Then its residue modulo N is zero if and only if the represented value is zero.*

Proof. Since N is odd, 2 is a unit modulo N . Hence

$$2^e M \equiv 0 \pmod{N} \iff M \equiv 0 \pmod{N}.$$

The hypothesis $|M| < N$ makes the latter congruence equivalent to $M = 0$. That is exactly the zero value; the two IEEE-754 signed zeros are deliberately identified by the algebraic semantics. $\square$

Nonzero collisions remain ordinary finite-modulus fingerprint collisions. If two distinct exact dyadic expressions on the same inputs compare equal, then after clearing powers of 2, their difference has numerator divisible by the chosen modulus. The suffix-2 variants exist so that a suspected collision can be checked against a different set of primes or composite moduli: `Field2`, `FieldFast2`, `SophieGermainRing2`, and `PythagoreanRing2` have entirely different nonzero collision sets.

5 How the Field primes are chosen

The statement “reduce modulo an odd prime” leaves many possible primes. The implementation chooses primes satisfying several constraints simultaneously. These constraints are motivated by different public semantics: some are needed by the faithful `Field` root operations, some by `FieldWithMulCasts` casts, and some by all Field-family modes. The prime tables are nevertheless shared by `Field`, `FieldFast`, and `FieldWithMulCasts` (and by their suffix-2 variants) so users can switch among these policies without changing the finite fingerprints of inputs.

5.1 Size and sentinel constraints

The prime should be close to 2^w , because a larger field gives a lower collision probability. It must still leave room for infinity and NaN:

$$p \leq 2^w - 2.$$

It must also be larger than the maximum possible $|M|$ for the format’s signed significand M , so that Proposition 1 applies and zero is hit only by true zero.

Constraint 1 (size, sentinels, and zero). *For a source format of width w , the Field prime p must satisfy*

$$M_{\max}(w) < p \leq 2^w - 2,$$

where $M_{\max}(w)$ is the largest possible $|M|$ in the representation $x = 2^e M$ for that format.

5.2 Algebraic roots

The faithful algebraic modes try to treat square root and cube root as algebraic power maps when the chosen modulus allows it, but the exact contract differs by semantics. In the Field family, Constraints 2 and 3 are used by `Field` and `FieldWithMulCasts`. `FieldFast` shares the same primes for compatibility, but deliberately represents roots by tagged fingerprints.

For `Semantics::Field`, the modulus is a prime p , so the unit group has order $p - 1$. The square-root power map uses the following condition.

Constraint 2 (square-root power map).

$$p \equiv 3 \pmod{4}.$$

This makes 2 invertible in the odd part of the unit-group exponent and gives the multiplicative square-root choice used below.

Under this condition,

$$S(x) = x^{(p+1)/4}$$

is a multiplicative square-root choice:

$$S(xy) = S(x)S(y).$$

It is a true inverse to squaring on the square residues and gives a consistent choice on the nonsquares. No multiplicative square root can be a two-sided inverse on all of $\mathbb{F}_p^\times$, since squaring is two-to-one.

For cube roots, the power map $x \mapsto x^r$ is a perfect cube root exactly when $3r \equiv 1 \pmod{p-1}$. That requires $3 \nmid p-1$, i.e. $p \equiv 2 \pmod{3}$.

Constraint 3 (perfect cube-root power map).

$$p \equiv 2 \pmod{3}.$$

Equivalently, 3 is invertible modulo $p-1$, so a power map can be an inverse to cubing on $\mathbb{F}_p^\times$.

Together, Constraints 2 and 3 give

$$p \equiv 11 \pmod{12}.$$

That congruence is the root-related constraint on the Field primes.

The composite algebraic modes use the same power-map idea with the exponent of the unit group of $\mathbb{Z}/N\mathbb{Z}$. The Sophie Germain choices keep 3 invertible in that exponent, so `cbtr` remains a perfect power-map cube root. The Pythagorean choices cannot keep that property while also making trigonometric rotations available; there `cbtr` is represented as a tag. Square root remains a multiplicative power map in the faithful algebraic modes, although the round trip $\sqrt{x^2} = x$ holds only on the part of the ring selected by that power-map choice.

5.3 Multiplicative casts

The optional multiplicative-cast policy is the only cast policy here that tries to preserve algebraic structure. It is exposed as

`Semantics::FieldWithMulCasts.`

The ordinary `Semantics::Field` variants and `FieldFast` use the same primes and finite-value fingerprints, but cheap deterministic casts. Triton’s casts are signed integer resizes of the scrambled fingerprint, and the composite algebraic semantics use a deterministic finite-residue convention.

A cast between different Field widths cannot be additive, hence cannot be a ring homomorphism. Indeed, if $p \neq q$, every additive-group homomorphism $(\mathbb{F}_p, +) \rightarrow (\mathbb{F}_q, +)$ is trivial. The only nontrivial algebraic structure available across different characteristics is therefore multiplicative: the groups of nonzero residues.

The multiplicative construction below applies only to finite nonzero fingerprints. Zero is mapped to zero, and the infinity/NaN sentinels map to the corresponding sentinels. The `fp32↔fp64` edge uses Pohlig–Hellman discrete logs because a linear scan over a group of size about 2^{32} would be too expensive; the group orders are chosen to make this feasible.

Constraint 4 (computable cast logarithms). *The Field group orders used for cast logarithms, especially $m_{32} = p_{32} - 1$, must be smooth enough for the Pohlig–Hellman path used by the implementation.*

The multiplicative group $\mathbb{F}_p^\times$ is cyclic. The multiplicative-cast tower consists of the widths 4, 8, 16, 32, 64; the 6-bit format uses a standalone prime and is not part of this tower. Write p_i for the Field prime at a tower width and $m_i = p_i - 1$. For each format class α at that width, choose a primitive root $g_{i,\alpha}$ of $\mathbb{F}_{p_i}^\times$. There is usually only one such class; the important exceptions are the two fp8 families, *e4m3* and *e5m2*, and the two 16-bit families, *f16* and *bf16*. The implementation uses g_i for one class and g_i^3 for the other. Since the Field primes are 11 (mod 12), 3 is coprime to m_i , so g_i^3 is again a primitive root. Let

$$L_{i,\alpha} : \mathbb{F}_{p_i}^\times \rightarrow \mathbb{Z}/m_i\mathbb{Z}$$

be discrete logarithm to the base $g_{i,\alpha}$. In these coordinates, multiplication in $\mathbb{F}_{p_i}^\times$ becomes addition in $\mathbb{Z}/m_i\mathbb{Z}$.

The chosen primes satisfy

$$m_4 \mid m_8 \mid m_{16} \mid m_{32} \mid m_{64}.$$

Moreover, the factors introduced at successive steps are coprime to the factors already present. Equivalently, whenever $m_i \mid m_j$, the cofactor m_j/m_i is coprime to m_i , and stepwise cofactors between i and j are pairwise coprime.

Constraint 5 (multiplicative cast tower). *The Field primes in the cast tower must satisfy*

$$m_4 \mid m_8 \mid m_{16} \mid m_{32} \mid m_{64}, \quad m_i = p_i - 1,$$

and whenever $i < j$, the cofactor m_j/m_i must be coprime to m_i ; in particular, the stepwise cofactors from i to j are pairwise coprime.

Here and below, $i < j$ means that the i -bit tower format is narrower than the j -bit tower format. Put $c_{ij} = m_j/m_i$. The coprimality condition in Constraint 5 gives a Chinese-remainder section

$$\sigma_{ij} : \mathbb{Z}/m_i\mathbb{Z} \rightarrow \mathbb{Z}/m_j\mathbb{Z}$$

characterized by

$$\sigma_{ij}(a) \equiv a \pmod{m_i}, \quad \sigma_{ij}(a) \equiv 0 \pmod{c_{ij}}.$$

The widening (“up”) cast $U_{i\alpha,j\beta}$ and narrowing (“down”) cast $D_{j\beta,i\alpha}$ are the multiplicative maps defined in logarithm coordinates by

$$L_{j,\beta}(U_{i\alpha,j\beta}(x)) = \sigma_{ij}(L_{i,\alpha}(x)), \quad L_{i,\alpha}(D_{j\beta,i\alpha}(y)) = L_{j,\beta}(y) \bmod m_i.$$

For two distinct format classes α, β at the same width, the same logarithm-coordinate rule gives the automorphism

$$L_{i,\beta}(C_{i\alpha,i\beta}(x)) = L_{i,\alpha}(x).$$

Thus f16 and bf16, or $e4m3$ and $e5m2$, are not silently identified even though they share a bit width and a Field prime.

Proposition 2 (The Field cast tower). *For $i < j < k$, and for any choices of format classes at those widths, Field widening maps compose, Field narrowing maps compose, and narrowing after widening is the identity. Suppressing the format labels in the notation,*

$$U_{jk}U_{ij} = U_{ik}, \quad D_{ji}D_{kj} = D_{ki}, \quad D_{ji}U_{ij} = \text{id}.$$

Also $D_{kj}U_{ik} = U_{ij}$. In the opposite order, $U_{ij}D_{ji}$ is generally not the identity on $\mathbb{F}_{p_j}^\times$; it is the projection onto the chosen order- m_i subgroup.

Proof. The maps U_{ij} and D_{ji} are homomorphisms because they are additive in logarithm coordinates. The narrowing identities are just transitivity of reduction modulo $m_i \mid m_j \mid m_k$. The widening identity follows from uniqueness in the CRT description: stepwise widening from i to j to k gives a logarithm congruent to the original one modulo m_i , and congruent to zero modulo each step cofactor; because those cofactors are pairwise coprime, this is the same as being zero modulo the total cofactor m_k/m_i , which is the direct widening U_{ik} . Similarly, $D_{kj}U_{ik}$ has logarithm $\sigma_{ik}(a) \bmod m_j$, which is congruent to a modulo m_i and to zero modulo m_j/m_i ; by uniqueness this is $\sigma_{ij}(a)$, hence $D_{kj}U_{ik} = U_{ij}$.

The identity $D_{ji}U_{ij} = \text{id}$ follows because $\sigma_{ij}(a) \equiv a \pmod{m_i}$. Finally, $U_{ij}D_{ji}$ first forgets the logarithm modulo the cofactor c_{ij} and then chooses the CRT lift with cofactor coordinate zero, so it is a projection, not the identity in general. The coprime-cofactor hypothesis is exactly what is used here: it gives the CRT sections σ_{ij} , and it turns “zero modulo each step cofactor” into “zero modulo their product,” so stepwise widening agrees with direct widening. $\square$

6 Why finite fields cannot model exponential laws

In any finite field $\mathbb{F}_q$, with q any prime power, any group homomorphism

$$(\mathbb{F}_q, +) \longrightarrow (\mathbb{F}_q^\times, \cdot)$$

is trivial: the source has order q , while the target has order $q - 1$. Their orders are coprime. Thus a field-valued fingerprint cannot have a nontrivial function satisfying

$$\exp(a + b) = \exp(a) \exp(b)$$

as a function only of its field residue.

To model the exponential law, the fingerprint must remember another residue modulo a prime d for which the multiplicative side has an order- d subgroup, alongside a prime p whose unit group contains that subgroup. The Chinese Remainder Theorem keeps this compact. With $n = pd$,

$$\mathbb{Z}/n\mathbb{Z} \simeq \mathbb{F}_p \times \mathbb{F}_d.$$

The cost is algebraic: $\mathbb{Z}/n\mathbb{Z}$ has zero-divisors, so it is a ring rather than a field.

7 The Sophie Germain ring

`Semantics::SophieGermainRing` uses

$$n = pd, \quad p = 2d + 1,$$

with p and d prime. Strictly speaking, d is the Sophie Germain prime and p is the corresponding safe prime; the semantics name refers to the pair.

Since $d \mid p - 1$, the group $\mathbb{F}_p^\times$ contains a subgroup of order d . The implementation chooses an element $g \in (\mathbb{Z}/n\mathbb{Z})^\times$ whose $\mathbb{F}_p$ -component has order d and whose $\mathbb{F}_d$ -component is 1. It then defines

$$\exp(x) = g^{x \bmod d}.$$

This is a genuine homomorphism from the additive d -channel to the multiplicative order- d subgroup:

$$\exp(a + b) = \exp(a) \exp(b).$$

The logarithm is the dual map. It projects a unit to the order- d subgroup in the $\mathbb{F}_p$ -component, takes the discrete logarithm to base g , and returns the corresponding element of the additive order- d subgroup of $\mathbb{Z}/n\mathbb{Z}$. Thus, on the modeled image,

$$\log(xy) = \log(x) + \log(y), \quad \exp(\log(\exp x)) = \exp x.$$

The same channel carries other bases:

$$\exp_b(x) = g^{K_b x \bmod d}.$$

The constants K_b are fixed units modulo d . They are not numerical approximations to $\log_b e$; those constants are irrational and have no meaning as residues of dyadic values. The contract is that each base preserves its own law and has its own inverse.

Because the modulus is composite, division is exact only on units. If the denominator is a zero-divisor, the implementation returns the algebraic NaN fingerprint. At the sizes used here, zero-divisors are rare compared with the zero-divisors in Triton's $\mathbb{Z}/2^w\mathbb{Z}$, but they are real.

8 The Pythagorean ring

`Semantics::PythagoreanRing` uses

$$n = pd, \quad p = 4d + 1,$$

again with p and d prime. This still gives $d \mid p - 1$, so the $\exp/\log$ channel remains available. The new feature is that $p \equiv 1 \pmod{4}$, so -1 has a square root in $\mathbb{F}_p$. This allows a finite-ring analogue of complex rotations.

The implementation works in $(\mathbb{Z}/n\mathbb{Z})[i]$ with $i^2 = -1$. It chooses an element

$$\omega = \omega_{\text{re}} + i \omega_{\text{im}}$$

of order d and norm 1, and defines

$$\cos x = \Re(\omega^{x \bmod d}), \quad \sin x = \Im(\omega^{x \bmod d}).$$

Complex multiplication immediately gives

$$\begin{aligned}\cos(a + b) &= \cos a \cos b - \sin a \sin b, \\ \sin(a + b) &= \sin a \cos b + \cos a \sin b, \\ \cos^2 x + \sin^2 x &= 1.\end{aligned}$$

This variant gives up perfect cube roots. If $d > 3$ is prime and $p = 4d + 1$ is also prime, then $d \not\equiv 2 \pmod{3}$, since that would make $p \equiv 0 \pmod{3}$. Thus $d \equiv 1 \pmod{3}$, so $3 \mid d - 1$. In the small $d = 3$ case, $3 \mid p - 1$. In either case the unit-group exponent has a factor of 3, so the cube map is not globally invertible by a power map. `cbrt` is therefore represented as a tag in the Pythagorean semantics.

9 Tags and genuinely opaque functions

Not every operation has a useful algebraic law in the chosen finite model. For opaque `libdevice` calls, miscellaneous transcendentals, and Field-mode `exp/log/trig` calls, the implementation produces a deterministic tag. In this document, a tag means the resulting fingerprint, while the operation identifier or symbol hash is the salt used to construct it.

For a k -ary opaque operation symbol f , one should think of the implementation as choosing a deterministic map

$$\tau_f : R^k \longrightarrow R$$

for the current fingerprint ring R , extended to the sentinel values by the algebraic NaN rules. It promises congruence:

$$f = g, \quad a_i = b_i \text{ for all } i \quad \implies \quad \tau_f(a_1, \dots, a_k) = \tau_g(b_1, \dots, b_k).$$

It also uses the operation identity and argument order, so changing the symbol or changing the operands normally changes the fingerprint, up to ordinary finite-ring collisions. What it deliberately does not promise is an algebraic law such as $\log(xy) = \log x + \log y$, unless the semantics has a specific channel implementing that law.

This is not merely a weak fallback. Algebraically, an opaque call is best viewed as adjoining a fresh formal generator $t_{f,\mathbf{a}}$ for the operation and its argument tuple, then continuing the surrounding calculation in the chosen ring. A fixed-width sanitizer cannot literally adjoin infinitely many independent generators, so the tag is the compact residue fingerprint of that formal-generator idea. The same expression receives the same tag; unrelated opaque expressions are kept distinct except for the unavoidable collision risk of any finite fingerprint.

Triton uses the same opaque-operation idea for operations it does not give a structured algebraic model. For a fixed list of named unary operations, such as `log`, `sqrt`, `erf`, `floor`, and `ceil`, Triton applies a hash-like permutation of the already-scrambled payload, salted by the operation identifier. For arbitrary extern or `libdevice` calls, it uses a stable hash of the symbol name together with an argument-order-sensitive mix of the operand payloads. These are tags in the same semantic sense as above: deterministic, operation-distinguishing, and deliberately uninterpreted. `hip-fpsan` matches those formulas in `Semantics::Triton`; in the algebraic modes the concrete mixing formula changes to live inside the chosen residue ring, but the contract is the same.

No transcendence conjecture is involved in tags. They behave like uninterpreted operation symbols with congruence, not like real transcendental functions. No identity connecting $\tau_f(a_1, \dots, a_k)$ back to the operands is asserted in the first place.

Once produced, a tag is just another fingerprint. The ambient ring laws still apply to expressions built from it: for example $t + t = 2t$ and $(t + u)v = tv + uv$ are still enforced. The only thing withheld is a mathematical relation between t and the operands of the opaque operation.

10 Infinity, NaN, and order

The finite residue model is extended by one unsigned infinity and one absorbing NaN. In the Field case, the finite residues together with infinity form the projective line

$$\mathbb{P}^1(\mathbb{F}_p) = \mathbb{F}_p \cup \{\infty\},$$

and the full carrier of the semantics is

$$\mathbb{P}^1(\mathbb{F}_p) \cup \{\text{NaN}\}.$$

For the composite ring modes, read the same picture as the affine residue ring together with one formal point at infinity, again plus NaN. Correspondingly, the arithmetic rules model the projective line rather than IEEE-754-style signed infinities:

$$x/0 = \infty \quad (x \neq 0), \quad x/\infty = 0,$$

and indeterminate forms such as $0/0$, $0 \cdot \infty$, and ∞/∞ produce NaN. Since the infinity is unsigned, $\infty + \infty$ also produces NaN in this model. Arithmetic operations on finite residues never overflow; among the basic arithmetic operations on finite values, the only source of infinity is $x/0$ with $x \neq 0$.

No finite ring can carry a total order compatible with its addition and multiplication. Consequently the algebraic semantics do not model IEEE-754 numeric ordering. Comparisons, `min`, and `max` use deterministic fingerprint ordering, as Triton FPSan does.

11 What the suffix “2” means

The suffix-2 variants are independent choices of moduli under the same design constraints:

$$\begin{array}{lll} \text{Field2}, & \text{FieldFast2}, & \text{FieldWithMulCasts2}, \\ \text{SophieGermainRing2}, & & \text{PythagoreanRing2}. \end{array}$$

They exist because the residue map is a fingerprint, not an injection. If a check depends on a rare collision for one modulus, it should not depend on the corresponding collision for a second independently chosen modulus. Rerunning with the suffix-2 semantics is the compact version of rerandomizing the prime.

12 Extension to irrational number schemes

The construction above used one special fact about ordinary binary floating point: finite values are dyadic rationals, hence elements of the localization $\mathbb{Z}[1/2]$. But many logarithmic-number-system variants make irrational algebraic values exactly representable by allowing fractional binary exponents: half-integral exponents give $\sqrt{2}$, quarter-integral exponents give $2^{1/4}$, and finer grids go further. Such formats may or may not keep a mantissa. Multiplication is then exponent addition, while addition is the hard operation. For FPSan reduction, the only essential change is that the finite values now lie in a number field rather than in $\mathbb{Q}$.

In general, let K be a number field with ring of integers $\mathcal{O}_K$, and suppose the representable finite values lie in a localization $S^{-1}\mathcal{O}_K$, or in an order inside it. For every maximal ideal $\mathfrak{p}$ avoiding S , reduction gives

$$S^{-1}\mathcal{O}_K \longrightarrow \mathcal{O}_K/\mathfrak{p}.$$

The target is a finite field, not necessarily a prime field. Thus “reduce floats mod p ” is shorthand for reducing the localized ring of algebraic integers modulo a finite prime.

Consider the concrete quarter-exponent example. Let

$$\alpha = 2^{1/4}, \quad K = \mathbb{Q}(\alpha), \quad \alpha^4 = 2.$$

A toy format with scale factors $2^{k/4}$ and a binary mantissa has finite values in

$$\mathbb{Z}[1/2, \alpha] = \mathbb{Z}[\alpha][1/2].$$

Negative quarter-exponents introduce no new primes in the denominator, because

$$\alpha^{-1} = \alpha^3/2,$$

so inverting α only inverts 2. The polynomial $X^4 - 2$ is Eisenstein at 2, and the discriminant of the basis $1, \alpha, \alpha^2, \alpha^3$ is

$$\text{disc}(X^4 - 2) = 4^4 2^3 = 2^{11}.$$

Thus ramification is confined to 2. Every odd prime remains available for reduction, exactly as for dyadic rationals.

For an odd rational prime p , Dedekind’s theorem reads the decomposition of (p) from the factorization of $X^4 - 2$ over $\mathbb{F}_p$:

condition on p	$X^4 - 2$ over $\mathbb{F}_p$	residue fields
$p \equiv 1 \pmod{8}$, 2 a fourth power	$1 + 1 + 1 + 1$	$\mathbb{F}_p, \mathbb{F}_p, \mathbb{F}_p, \mathbb{F}_p$
$p \equiv 1 \pmod{8}$, 2 not a fourth power	$2 + 2$	$\mathbb{F}_{p^2}, \mathbb{F}_{p^2}$
$p \equiv 7 \pmod{8}$	$1 + 1 + 2$	$\mathbb{F}_p, \mathbb{F}_p, \mathbb{F}_{p^2}$
$p \equiv 3 \pmod{8}$	$2 + 2$	$\mathbb{F}_{p^2}, \mathbb{F}_{p^2}$
$p \equiv 5 \pmod{8}$	4	$\mathbb{F}_{p^4}$

Here $1 + 1 + 2$ means two linear factors and one irreducible quadratic factor. The first line is the split case. The last line is the inert case: p remains prime in $\mathbb{Z}[\alpha]$, and the residue field is the non-prime finite field $\mathbb{F}_{p^4}$. The degree-two cases similarly produce residue fields $\mathbb{F}_{p^2}$. Only the completely split case lands entirely in copies of $\mathbb{F}_p$.

For fingerprinting, any of these prime ideals can be a target. If its residue degree is f , the finite carrier is $\mathbb{F}_{p^f}$, and the leaf map sends

$$m \alpha^k 2^{-r} \longmapsto \overline{m} \overline{\alpha}^k \overline{2}^{-r} \quad \text{in } \mathbb{F}_{p^f}.$$

Since p is odd, $\overline{2}$ is invertible, so the scale factors remain units and the same zero-safety argument applies. This is not implemented here, but the algebraic mechanism is unchanged: ordinary binary floats use the number field $\mathbb{Q}$; quarter-exponent LNS uses $\mathbb{Q}(2^{1/4})$; other algebraic scale systems use other number fields. Maximal ideals not inverted by the format still supply finite fields in which exact algebraic identities can be fingerprinted.